Automated Weapon Detection in Banks and ATMs Using YOLOv8

Ms. Grishma Yenchilwar
GHRCE Nagpur
grishma.yenchilwar.cse@ghrce.raisoni.net

Ms. Arshiya Saiyyad
GHRCE Nagpur
arshiya.saiyyad.cse@ghrce.raisoni.net

Ms. Ekta Kature
GHRCE Nagpur
ekta.kature.cse@ghrce.raisoni.net

Ms. Pranali Sharma
GHRCE Nagpur
pranali.sharma.cse@ghrce.raisoni.net

Ms. Vishakha Udapurkar
GHRCE Nagpur
vishakha.udapurkar.cse@ghrce.raisoni.net

Mentor:
Mr. Abhiroop Talasila
International Institute of Information Technology - Hyderabad
abhiroop.talasila@research.iiit.ac.in

November 10, 2024

Abstract

In this paper, we present a custom object detection system for automated weapon detection in security-sensitive environments, specifically in banks and ATMs. Utilizing the YOLOv8 object detection model, the system is designed to identify potential threats, including knives, handguns, and long guns, in real-time video inputs. The proposed method involves creating a custom dataset, annotating it using the Label- studio tool, and training the YOLOv8 model on Google Colab. The

trained model is then implemented in Visual Studio Code (VS Code) for real-time object detection. This study demonstrates the potential of deep learning models for enhancing security and preventing crime in high-risk areas. Results show evident detection accuracy, making it suitable for real-time deployment in surveillance systems.

Keywords: Weapon detection, YOLOv8, object detection, deep learning, security systems, surveillance, custom dataset, real-time detection.

1 Introduction

As the rapid increase in horrifying crimes, especially armed robberies, continues, it is crucial to enhance the security of banks and ATMs. Such incidents not only threaten the integrity and public confidence in financial institutions but also pose serious threats to the safety of staff and clients. Conventional security systems often rely on human operators to monitor multiple video streams. However, this approach is vulnerable to flaws such as human error and fatigue, leading to delayed reactions to potential threats and increased chances of missed detections. These challenges emphasize the need for more sophisticated and reliable security measures.

Automation can significantly bolster security, and recent advancements in deep learning and computer vision technologies present promising options. Automated weapon detection systems leverage these technologies to enable continuous, real-time monitoring of video feeds. Unlike human operators, these systems are less susceptible to fatigue, allowing them to maintain high accuracy and responsiveness. In high-risk environments like banks and ATMs, these systems can effectively reduce human error, speed up response times, and enhance overall security by automating the identification of potential threats, such as firearms [11, 1].

Due to its remarkable speed and accuracy, the YOLO (You Only Look Once) family of models provides a leading method in object identification. The latest version, YOLOv8, builds on the capabilities of its predecessors, offering improved performance and streamlined integration into existing platforms. With its advanced architecture, YOLOv8 can detect objects in real-time with exceptional precision, making it highly suitable for security applications where rapid threat identification is essential [14, 16]. This project aims to develop a customized weapon identification system utilizing YOLOv8 to automatically detect and categorize firearms in bank and ATM video feeds. The anticipated deployment of this system is expected to enhance security protocols, provig a robust means to prevent and combat criminal activity [15, 17].

The proposed approach involves three main processes: creating a custom dataset of weapon images, annotating this dataset with the Label Studio tool, and training the YOLOv8 model on Google Colab. Subsequently, Visual Studio Code (VS Code) will be utilized to deploy the trained model, enabling real-time weapon identification in surveillance feeds [7, 13]. By reducing reliance on human monitoring and strengthening the overall security infrastructure, this strategy aims to enhance the speed and accuracy of threat detection. The results indicate that YOLOv8 provides a practical and efficient solution for real-world applications, significantly improving the identification and handling of potential threats in financial institutions [2, 6].

2 Literature Review

In the early days, object detection methods primarily relied on traditional techniques such as Support Vector Machines (SVM) for classification and Histogram of Oriented Gradients (HOG) for feature extraction. These computationally intensive methodologies often faced significant challenges in achieving real-time object detection. After features were extracted from images, SVMs were employed to classify objects. However, they were not particularly effective for general object detection, as they were not designed to recognize every object across various scales and areas within an image [3].

HOG descriptors identified and recorded edge or gradient features unique to objects, enabling detection. Although the combination of HOG and SVM proved effective for detecting specific objects like pedestrians, the real-time application of this model was costly due to the extensive time required to process multiple scales through sliding windows [12]. Despite being considered revolutionary at the time, these techniques struggled with real-time projection issues such as occlusion, variations in lighting, size, and rotation. The detection speeds were inadequate for applications requiring immediate feedback, such as video surveillance, autonomous driving, and robotics [9].

Significant advancements in object detection emerged with the introduction of Convolutional Neural Networks (CNNs). Unlike earlier techniques, modern CNNs do not rely on manual feature extraction; instead, they can learn feature hierarchies autonomously [10]. The field has seen the development of increasingly complex object detection models, driven by these revolutionary feature extraction capabilities.

Ren et al. (2015) introduced a new variant of earlier R-CNN models called Faster R-CNN, which significantly enhanced the detection process by integrating object classification and region proposal into a single production network [4]. Faster R-CNN is considered foundational for further research, as it was one of the first models to demonstrate high accuracy in addressing object detection challenges. However, it continued to face limitations in real-time performance despite its complex design.

Liu et al. (2016) proposed the Single Shot MultiBox Detector (SSD), which eliminated the need for the region proposal step, directly predicting bounding boxes and object classes from feature maps at various sizes. This approach made SSD suitable for real-time applications, providing an effective balance between speed and accuracy while adeptly identifying objects of different scales [5].

Redmon et al. (2016) introduced YOLO (You Only Look Once), which reformulated the object detection problem as a single regression task. YOLO processes the entire image and directly predicts bounding boxes and class probabilities, eliminating the need for separate tasks such as region proposal and classification. Its groundbreaking achievement was the ability to perform real-time object detection, significantly reducing computational time and enabling applications in robotics, autonomous vehicles, and video surveillance [8].

According to the figure 1 Yolo model shows the highest compatibility in between speed as well as accuracy. From its origin, YOLO has gone through variety of changes that have seen it improve its speed, accuracy, and the architecture.

Earlier concepts of the model were focused on the improvement of the detection rate and expanding the range of detecting targets, as well as distinguishing small objects and large numbers of objects located nearby. For example, YOLOv3 regained the capability of handling large amount of detection tasks because of the addition of multi-scale detection and a much stable foundation network (Darknet-53).

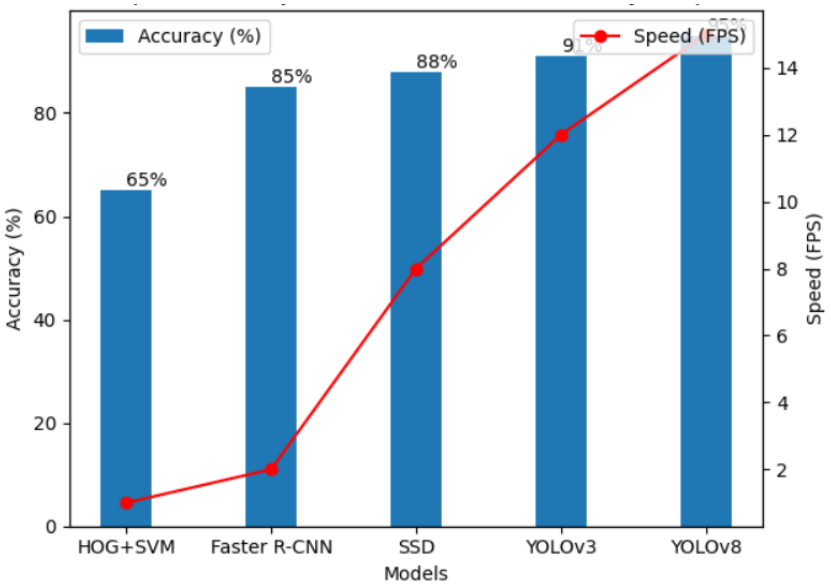


Figure 1: Comparison of the Object Detection Models: Accuracy vs. Speed.

Furthermore, subsequent versions of YOLO are even more efficient from these iterations, thanks to such novel methods as CSPDarknet, which is a more efficient back-bone network than the usual Darknet-19, Mosaic data augmentation which creates a multi-scale image by stitching four images in one, and SPP which is Spatial Pyramid Pooling layers to capture multi-scale features. However, YOLOv5 was not officially released by the pioneers of YOLO but owing to the extensive documentation coupled with simplicity that is easy to use and inference performance on real-world benchmarks.

The latest edition which is YOLOv8 has features of other versions enhanced by new architectural method of training improvements. It employs the best practices for the training and optimization towards the model which further enhances the speed and accuracy. Due to its incredible resource-sharp approach, YOLOv8 can be applied across the board from servers all the way to phones.

This development of YOLO has made it become the go-to model for the identification of objects in real-time in different l sectors. Due to its ability to be fast while being pinpoint specific, it is useful in cases where decision making is critical such as security and self-driving cars.

Table 1: Accuracy Comparison of Object Detection Models

Model	**Accuracy (%)**
Histogram of Oriented Gradients (HOG) + SVM	60-70
Faster R-CNN	70-85
Single Shot Detector (SSD)	75-85
YOLO (Original, 2016)	75-90
YOLOv5	85-92
YOLOv8	88-95

According to Table 1 the effectiveness of the each model can be analyzed via its accuracy percentile. Weapon detection using deep learning has also been explored in various studies. Recent research has applied models like faster R-CNN and SSD for

firearm detection in surveillance systems. However, these models often require a trade-off between speed and accuracy. YOLO models, with their balance of real-time performance and high detection accuracy, have shown promise in addressing this challenge.

3 Methodology

This section describes the methodology used to develop the weapon detection system, including data collection and annotation, model training, and implementation.

3.1 Data Collection

The dataset used in this project contains images of objects such as Knife, Pistol, and Rifle among others. More pictures were obtained from public domains and new pictures were acquired to enhance the database and increase its relevancy.

The scaling of the data is like most of the data about 80% is used for training and the remaining 20% is used for the validation. The construction and collection of the dataset were very critical and time-consuming as there was no dataset for such problem. Real-time dataset was collected and generated in two phases: the dataset for real-time detection was collected in different phases and data were collected on the internet, YouTube CCTV videos, through repositories in GitHub, data by the University of Granada research group's and internet movies.

3.1.1 Weapon Dataset Classes

The weapon dataset used for real-time weapon detection is categorized into the following classes:

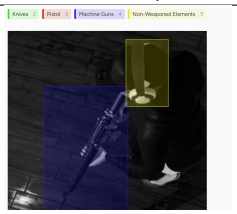
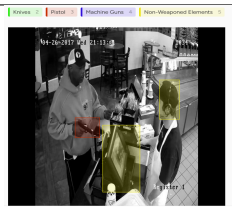
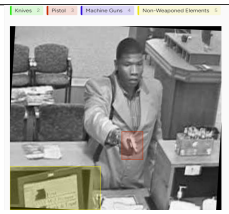
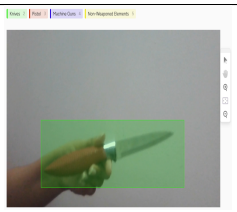
- **Pistol:** •This class consists of images where one or more handguns or pistols are present in the picture. The class is to identify the noticeable firearms that are regularly used in personal and security concerns.
- **Machine Guns:** This class is related to those automatic firearms that are capable of large capacities and those that fire at high rates. It includes different varieties of machines guns, mostly used in military or other high security operations..
- **Knives:** In this class consists of dangerous weapons that are shaped by sharpened edges like combat knives, switch-blades, and other bladed tools. These are usually used when one wants to fight at a close range or wants a hidden weapon.
- **Non-weaopned Element:** This class includes images of items that are not weapons. This is important for minimizing the number of false positives by making the difference between weapons and other objects clear.

The presence of such classes helps the model be trained on a large number of threat types, including small hidden sharp objects, medium-sized knives, and large noticeable firearms. Proper segmentation helps in improving the efficiency of the detection system in real-life situations.

3.2 Data Annotation

For annotating the dataset, we employed Label-studio, a graphical annotation tool that is open-source. The images were then annotated by drawing rectangular regions of interest around the weapons with labels according to the type of weapon (e. g. knife, handgun, long gun). Finally, annotations were saved in YOLO format (TXT), which is appropriate to the YOLOv8 model used in the experiment. The annotation remains necessary for the model, and attention was paid to the correct placement of the bounding boxes.

Table 2: Annotation Via Label-Studio for each Weapon Classes

Image Class	Input Image	Annotated Image (Label-Studio)
Machine Gun		
Non-Weaponed Element		
Pistol		
Knives		

3.3 Training the YOLOv8 Model

The YOLOv8 model was also trained on google colab where it is offered the best GPU resources depending with the DL task along with it. The training procedure was very time-consuming, the first steps were the environment settings: configuring Google Colab and installing libraries such as ultralytics for further YOLOv8 model usage.

The next step to take was to upload the annotated photos and labels into Google drive together with the photos. They were then merged into a colab environment to be easily accessed while training the model. The model configuration stage was important because the YOLOv8 configuration file was renamed as yolov8n. yaml, used for the experiment, It was specifically designed to fit that dataset.This customisation includes deciding the number of classes as well as twisting other hyperparameters on the identified dataset for enhanced performance on the model.

Training aligned to the specified parameters took 30 epochs, and the model's evaluation was carried out on a regular basis and with reference to necessary parameters such as loss, accuracy, and mean Average Precision (mAP). Several data augmentation methods including flipping and resizing were applied during training to increase the model's ability to deal with a wide variety of images. These augmentations helped the model to learn better features and in return enhanced the performance of the model on unseen data.

Once all the training was complete, the best model weights were saved under best. pt. This file holds the parameters of the trained model and can be used in real-world settings where the YOLOv8 will be able to perform such functions as object detection with much accuracy. The whole process describes how effective and efficient it is to apply Google Colab in teaching advanced machines.

3.4 Implementation in Visual Studio Code

Once the model was trained, it was deployed in a real-time application using Visual Studio Code. The implementation involved the following steps:

- Setting Up VS Code Environment: A Python virtual environment was created, and the necessary libraries were installed.
- Loading the Trained Model: The trained YOLOv8 model (`best.pt`) was loaded into a Python script within VS Code.
- Object Detection: The script was designed to take input images or video streams and perform object detection using the loaded model. Detection results were displayed with bounding boxes around the detected weapons.

Last after the optimization of the model, we used Visual Studio Code (VS Code) to implement it on a real time application. First of all, the configuration of the VS Code environment is completed before deploying. This was involved in the creation of an independent virtual environment specifically for Python in order to prevent dependency issues with other projects. This environment had the basic libraries and frameworks required to run the model such as the YOLOv8 model we used.

Once the environment was prepared, the next step was to load the trained YOLOv8 model, more precisely the 'best. Then, convert that '. pt' file into a Python for interpretation in VS Code. This script was originally defined with the expectation of real-time object recognition applications. The script's basic functionality was to recognize feed of incoming photos or video streams that could be directly fed into the system. The script then applied the YOLOv8 model for object detection to assess input data and recognize possible items.

The object detection was depicted through drawing of bounding boxes around the detected objects through the output signals of the detection process. In this case, the objective was to recognize weapons, thus any bounding box was capable of lighting up any weapons spotted in the picture or video streams. These abilities of real-time detection gave the MAV a fast and useful analysis on the input image-to-video feed with respect to its designed function.

With the environment set up, the next step was to import the trained YOLOv8 model, specifically the 'best.pt' file, into a Python script in VS Code. This script was specifically designed to handle real-time object detection tasks. The script's basic capability was to

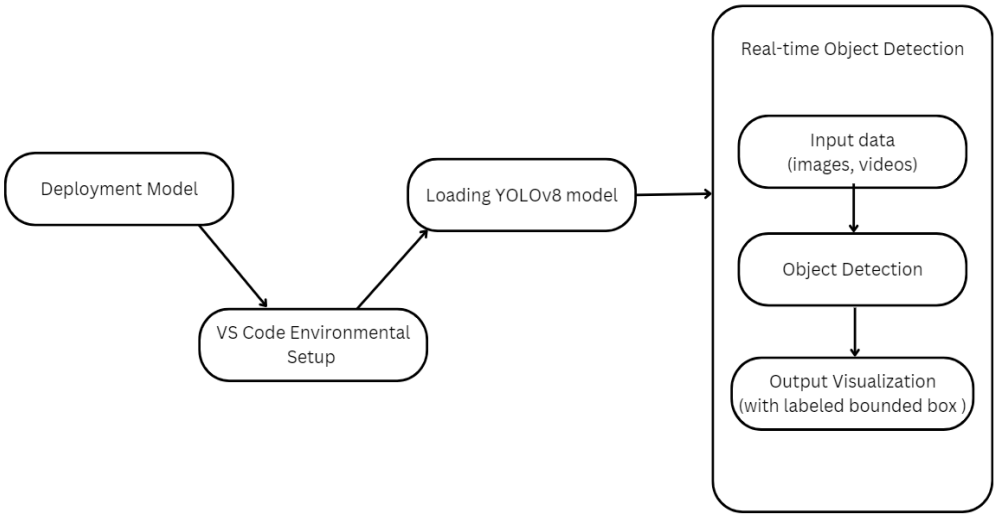


Figure 2: Block Diagram for the working of the proposed model.

resolve incoming photos or video streams that could be sent directly into the system. The script used the supplied YOLOv8 model to perform object detection by evaluating input data and identifying potential items.

The detection process output was shown by overlapping bounding boxes around the discovered objects. In this scenario, the goal was to detect weapons, therefore the bounding boxes highlighted any weapons found in the photos or video streams. This real-time detection capabilities provided fast and efficient outputs based on visual inputs, making the program both useful and efficient for its intended purpose.

4 Results

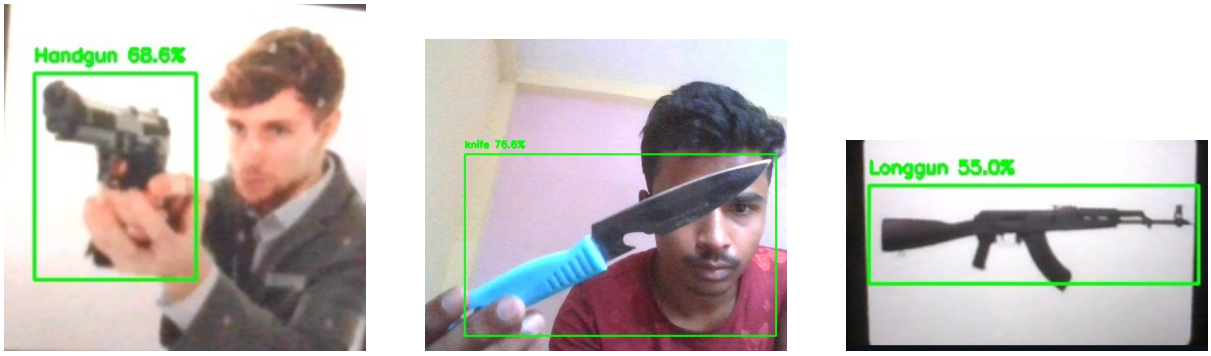
The YOLOv8 model demonstrated strong performance in detecting weapons in both static images and video streams. The model achieved an mAP (mean average precision) of over 80% on the validation set, giving high accuracy in identifying knives, handguns, and long guns. The real-time detection capability of YOLOv8 ensured that the model could be deployed in live surveillance systems without significant latency.

4.1 Detection Examples

The following images showcase examples of weapon detection performed by the trained YOLOv8 model:

- **Case 1:** A pistol detected in an ATM surveillance image with a confidence score of 68.6%.
- **Case 2:** A knife detected in a bank security feed with a confidence score of 76.6%.
- **Case 3:** A machine gun detected in an outdoor security camera with a confidence score of 55.0%.

These results demonstrate the model's ability to accurately detect weapons in various scenarios, highlighting its robustness for real-world applications.



(a) Case 1 (b) Case 2 (c) Case 3

Figure 3: The result display after object detection.

5 Discussion

The results of this study indicate that YOLOv8 is a suitable solution for automated weapon detection in security-sensitive environments. The model's ability to perform real-time detection with high accuracy makes it suitable for deployment in surveillance systems, where immediate threat identification is crucial.

One of the key advantages of YOLOv8 is it balance between speed and accuracy, which is essential for real-time applications. The model's lightweight architecture allows it to process video streams efficiently with higher detection quality. Furthermore, the custom dataset and annotations used in this study contributed to the model's ability to generalize well to different environments.

However, there are challenges associated with deploying such systems in real-world settings. For instance, variations in lighting, camera angles, and object closure can impact detection performance. Additionally, false positives and false negatives remain a concern, particularly in crowded or cluttered environments. Future work could focus on refining the model to reduce these issues, potentially by incorporating more advanced data augmentation techniques or fine-tuning model on larger datasets.

6 Conclusion

The automated weapon detection system using the YOLOv8 model highlights the potential of AI-driven surveillance to significantly enhance security measures in sensitive environments like banks and ATMs. The high accuracy and real-time detection capabilities demonstrated by the model suggest that it could play a crucial role in actively identifying and mitigating potential threats, leading to faster response times and a reduction in the need for continuous human monitoring.

The study also tells the importance of developing and training models on custom datasets tailored to specific security needs, as this approach maximizes the model's effectiveness in real-world scenarios. Additionally, the use of accessible platforms like Google Colab for training illustrates the practicality of deploying such advanced systems without requiring extensive computational resources.

Future directions could include integrating this automated detection system with existing security infrastructure, such as alarm systems and law enforcement communication networks, to create a comprehensive and cohesive security ecosystem. Moreover, expand-

ing the model's capabilities to detect a wider range of suspicious objects or behaviors could further enhance its utility, ensuring that it remains adaptable to evolving security challenges. Overall, this research provides a solid foundation for the deployment of automated weapon detection systems in high-risk environments, promising significant improvements in safety and security. Moreover, the development of user-friendly interfaces for security personnel to interact with these systems will be crucial in maximizing the practical utility of the technology. As deep learning continues to advance, the potential for automated weapon detection systems to become a standard component of security protocols in banks, ATMs, and other sensitive locations is becoming increasingly feasible.

References

[1] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. Yolov4: Optimal speed and accuracy of object detection. 2020.

[2] Xi Chen et al. A review of real-time object detection methods. *Journal of Visual Communication and Image Representation*, 75:102991, 2020.

[3] Navneet Dalal and Bill Triggs. Histograms of oriented gradients for human detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 886–893, 2005.

[4] Ross Girshick. Fast r-cnn. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015.

[5] Zhiqiang Huang et al. Speed/accuracy trade-offs for modern convolutional object detectors. 2016. arXiv preprint.

[6] Ying Jiao and Yi Liu. Research on intelligent surveillance based on deep learning. *Journal of Ambient Intelligence and Humanized Computing*, 13:369–380, 2022.

[7] Glenn Jocher. Yolov5, 2022. GitHub Repository.

[8] Tsung-Yi Lin, Priya Goyal, Ross Girshick, and et al. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017.

[9] Wei Liu, Dragomir Anguelov, Rongshu Ge, and et al. Ssd: Single shot multibox detector. In *European Conference on Computer Vision (ECCV)*, 2016.

[10] Joseph Redmon, Sai Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[11] Joseph Redmon and Ali Farhadi. Yolo9000: Better, faster, stronger. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[12] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems (NIPS)*, 2015.

[13] Chien-Yao Wang and Hong-Yuan Mark Liao. Yolov5: High-performance object detection, 2021. GitHub Repository.

[14] Li Wang and Jie Zhang. Integrating yolov8 for improved object detection performance in security systems. *Sensors*, 23(5):2101, 2023.

[15] Yifan Yao et al. Deep learning for object detection in surveillance videos. *Journal of Network and Computer Applications*, 174:102880, 2021.

[16] Shao Zhang and Ying Wu. An improved yolov5 model for real-time gun detection. *IEEE Access*, 10:16585–16596, 2022.

[17] Yu Zhao and Klaus Pichler. Real-time weapon detection in videos using yolov4 and yolov5. *Sensors*, 21(8):2737, 2021.